后训练学习计划 • 第 7 周（Phase 2 • 10/19-10/25 • 数据工程：合成、清洗、发布自有数据集）

本周定位：**后训练岗位的真实日常**。你上周用别人洗好的 3000 条指令集，这周开始你自己造、自己洗、自己发布。产出是一份公开的「英语教学对话」指令数据集（名称含 `english-tutor-instruct` ）+ data card + 一条可复现的过滤管线 + 第 2 篇博客。**这份数据集是 M2/M3/M4 的原料**：周 8 的偏好对从它的输出里来，周 9 的可验证任务、周 10-12 的 Tutor 模型 SFT 都用它。 用法说明：每天先看「今日总览」，再从任务 1 逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗 = 硬性要求**，灰色小字块 = 注释（可跳过，不影响主线）。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一 10/19	清周 6 gaps + self/evol-instruct 方法 + 任务 taxonomy 与种子池	`taxonomy.md` （5 类 ≥40 模板）+ 匿名化种子 500 词	6h
Day 2 周二 10/20	生成管线打通：**API 算账与限额** + 200 条小样 + 演化	`cost.md` 外推表 + 缓存可续跑 + 200 条小样	6.5h
Day 3 周三 10/21	全量生成 4000 条 + 规则过滤 + PII 脱敏	`raw_4k.jsonl` + 规则命中表 + `PRIVACY.md`	6.5h
Day 4 周四 10/22	四板斧后两板斧：MinHash 去重 + 困惑度过滤 + RM 打分	三份过滤报告 + `pipeline.sh` 两次运行同 md5	6.5h
Day 5 周五 10/23	去污染 + 分布统计与图 + data card	`stats.json` + `dist.png` + README 草稿	6h
Day 6 周六 10/24	发布 ModelScope/HF + 复现包 + 第 2 篇博客	公开数据集链接 + 博客链接	6h
Day 7 周日 10/25	补漏 + 闭卷复测 + 7 项总验收 + 周 8 预习	验收 ≥6/7 项通过	4.5h

注：**本周预算与卡时**：API ≤¥30（唯一的大头，Day 2 第一件事就是算账，见任务 1 的硬闸门）、**卡时上限 4h ≈¥8**（PPL 过滤 1.5h + RM 打分 1h + 机动 1.5h）。数据工程主要吃 CPU 和网络，不开卡也能干半天，这是它和前面几周最不一样的地方。**超了怎么办**：把目标规模从 4000 条降到 1500 条（Day 3 的判据同步改成 ≥1400），并只保留 r 一档 RM 打分；不许砍过滤环节来换条数——**1500 条干净数据比 4000 条脏数据值钱，这是本岗的第一常识**。

注：本周最大风险是 **API 花费失控**（表现：脚本没限流、失败重试把同一批 prompt 打了三遍）。对策是三道闸：单次任务上限、按 hash 缓存（重跑不重复计费）、**当天累计花费写进日志并在 >¥20 时自动停**。第二风险是**你造出一堆自己都不愿意看的样本**（Day 2 的小样人工检就是拦这个的）。第三风险是发布环节的网络（hf-mirror 不能上传，见 Day 6 注）。

注：砍单顺序（实在做不完按这个砍）：Day 4 任务 3 的 RM 打分（降级为 LLM-as-judge，几乎不占卡）→ Day 5 任务 2 的分布图（四宫格砍成两张图）→ Day 6 任务 2 的 HF 补传（只发 ModelScope，HF 记 gaps 推到周 9）。**不许砍**：Day 2 的算账闸门、Day 3 的脱敏、Day 4 的去重与去污染（Day 5 任务 1）、Day 5 的 data card、Day 6 的发布。**许可与隐私是硬门槛：宁可发布 800 条，不可少写一行许可。**

二、开始前：账号、核价、依赖与目录（约 50 分钟，10/18 前一晚做完）

1. 选一个**按 token 计费、有 OpenAI 兼容接口**的国内大模型 API（Day 2 任务 1 要算账，所以今天必须拿到价格页）。充值 ¥40（本周上限 ¥30，留 ¥10 余量）。**今天只做两件事：注册 + 把价格页的输入/输出单价抄进 `week07/cost.md`，标明页面日期**。不要今天就开始打请求。
2. 本地依赖：

```
source E:/study/posttrain/.venv/Scripts/activate
pip install -U datasketch datasets pydantic matplotlib tqdm requests langdetect
python -c "import datasketch, pydantic, langdetect, matplotlib; print('DEPS OK')"
```

3. 目录与仓库：

```
mkdir -p E:/study/posttrain/week07/{day1,day2,day3,day4,day5,day6}
mkdir -p E:/study/llm-posttraining-lab/dataset-english-tutor/{scripts,data,card,figures}
```

4. 明日记忆侧只读准备（**功能开发仍然冻结，本周不写一行产品代码**）：确认能只读访问 `E:/CC/tomorrow-memory/`，Day 1 要从它的本地词库与 SQLite 里取种子。取数一律走 `sqlite3 ... "SELECT ..." > file`，**不要开服务、不要连生产库**。

> 注：`pip install datasketch` 会带 scipy，Windows 上偶尔与 conda 的 MKL 打架报 `OMP: Error #15: ... libiomp5md.dll already initialized`。正解一直是**用独立 venv 而不是 conda base**；急着往下做就先 `set KMP_DUPLICATE_LIB_OK=TRUE`（cmd）或 `export KMP_DUPLICATE_LIB_OK=TRUE`（Git Bash）绕过，然后把「切 venv」记进 gaps。这条你从第 1 周踩到现在，今天该彻底解决。

Day 1 • 周一：方法、任务拆分与匿名化种子池

今日总览（6h）：今天不写生成代码。你要交付三样纸面上的东西：清掉周 6 的洞、一份能指导整周的任务 taxonomy（5 类教学任务 × 若干指令模板）、一个**已经匿名化的 500 词种子池**。明天的 API 调用能不能省钱、出不出脏数据，全看今天的 schema 定得清不清楚。

任务 1：清周 6 gaps（约 60 分钟）

打开 `week07/gaps-from-week6.md`（周 6 Day 7 写的），本周 Day 1-2 只清这三条：

从上周带来的洞	本周哪天闭合	动作
只用过别人洗好的指令集，没做过数据生产与清洗	今天定 schema，Day 2-4 建管线	亲手生成 + 四板斧过滤，每步留命中数
多轮对话数据没造过（上周是单轮 alpaca 形态）	今天任务 3	每个任务类至少 3 条 2-4 轮的多轮模板
不知道合成数据的提示词怎么版本化与复现	Day 2 任务 2	提示词版本号进文件名 + seed/温度写死 + 请求 hash 缓存

完成判据：三条各有 `[x]` 或落到本周具体某天；`runs.csv`（沿用周 6 那张）新增一行 `week07-00` 记录今天的 schema 版本。

任务 2：self-instruct 与 evol-instruct 精读（约 90 分钟）

1. 读 self-instruct 的方法节：从少量人工 seed 任务出发，让 LLM **自举生成新指令 → 过滤 → 回灌**。记下它的过滤手段（与 seed 任务的 ROUGE 相似度、数字重合度等）。
2. 读 evol-instruct（WizardLM）的方法节：**深度演化**（给已有指令加约束、加具体性、要求多步推理、复杂化输入）与**广度演化**（同领域造一个全新任务、换格式、换角色）。各抄 3 个演化操作原文。
3. 写一段自己的话回答：**为什么"指令太像"比"指令太少"更致命？**（答到点上：数据多样性决定模型能覆盖的行为空间，同质数据只是把已有能力重复强化并加剧遗忘——周 6 的遗忘实验就是证据。）

完成判据：NOTES 新增 ≥12 条；能不看资料各说出 2 个深度演化和 2 个广度演化的具体操作；上面那段回答 ≥80 字且不含"提升质量"这类无法验证的词。

任务 3：教学任务 taxonomy 与指令模板（约 120 分钟）

明日记忆的 Tutor 路径输出的是一份结构化内容（`packages/agent/src/nodes/tutor.ts` 里定义的 mnemonic / wordRoot / sceneExamples / confusable 四块，输入含单词、用户 CEFR 等级、最近学习的 10 个词）。本周就围绕它设计 5 类任务：

类	任务	输入字段	期望输出 schema
T1	简明释义 + 词根	word, cefr	`{"gloss_zh":str,"word_root":str}`
T2	记忆口诀	word, cefr, recent_words	`{"mnemonic":str}`
T3	场景例句	word, scene_type	`{"scene_examples":[{"en":str,"zh":str,"scene":str}]}`
T4	易混词辨析	word, confusable_word	`{"diff":str,"pick_guide":str}`
T5	难度分级	word, gloss	`{"cefr":"A1|A2|B1|B2|C1|C2","reason":str}`

T5 现在多写一行注释：它是周 9/周 11 可验证奖励的来源（分类对不对是可以自动判的），所以 T5 的标签必须来自词库自带的 CEFR 字段，不许让模型自己标。

写 `dataset-english-tutor/card/taxonomy.md`：每类 ≥8 条指令模板（措辞、句式、长度、角色都要有变化，例：直接问 / 带上下文问 / 让模型纠错 / 限定输出格式 / 多轮追问），并给每类 1 条**你自己手写的金样例**（不许用模型生成，这是后面判断模型输出好坏的尺子）。

完成判据：5 类 × ≥8 模板 = **≥40 条**模板入库；每类有 1 条手写金样例；模板里凡约束输出格式的都写出可校验的 JSON 字段名（Day 3 的规则过滤器直接读这张表）。

> 注：金样例必须你自己写，别丢给模型再"改改"。周 8 你要拿它们当评分参照、周 12 当评测锚点，模型生成的参照会把它自己的偏差一起传下去——这类错误在数据团队里叫"用模型的输出给模型的输出标定"。

任务 4：种子池构建与匿名化（约 60 分钟）

1. 词表来源（本地，零成本、零许可风险）：`E:/CC/tomorrow-memory/ec-cedict.json` 与 `stardict.db`，抽 500 个带 CEFR 标注的词（按等级分层：A1/A2/B1/B2/C1 各 100）。**随机种子写死**（`seed=42`），并把抽样命令记进脚本。
2. 历史查询词来源：产品 SQLite 里的 `fsrs_cards` 表（字段含 `user_id, word, ...`）与 `tutor_cache` 表（`word, level, content`）。**只取 `word` 与 `level` 两列**，按词去重；`user_id`、`created_at` 一律不读、不写。
3. 写 `scripts/extract_seeds.py`（只读本地库；下面是取词部分，`<你的库路径>` 换成实际文件）：

```
python - <<'PY'
import sqlite3, json, collections
db = sqlite3.connect("file:<你的库路径>/data.db?mode=ro", uri=True)      # 只读模式，防误写
rows = db.execute("SELECT word, level, content FROM tutor_cache").fetchall()
buckets = collections.Counter()
out = []
for word, level, content in rows:
    if not (word and level):
        continue
    buckets[level] += 1
    out.append({"word": word.strip().lower(), "cefr": level,
                "cached_content": content})                              # 只留词+等级+已生成内容
json.dump(out, open("week07/day1/seeds_from_cache.json", "w", encoding="utf-8"), ensure_ascii=False)
print("cache rows:", len(out), "| level dist:", dict(buckets))
PY
```

4. 写 `dataset-english-tutor/card/PRIVACY.md`：字段白名单（只允许 word/cefr/生成内容）、删除项（`user_id`/时间戳/任何自由文本）、**明确禁止把 `chat_messages` 表的任何内容用于数据集**（那张表是用户与助手的自由对话，包含不可控的个人信息，脱敏成本高、收益低）。

完成判据：`seeds.jsonl` ≥500 行且**每行只有 word/cefr（可选 cached_content）三个键**（写 5 行代码断言键集合）；5 个等级的分布数字打印出来并入库；`PRIVACY.md` ≥200 字含白名单、删除项、禁用表三块。

> 注：`tutor_cache` 里存的是**历史上真实用户的查询产生的答案**。词本身（"ambiguous"）不识别个人，但 `content` 是一段 LLM 生成的解释，可能提到用户水平或场景。处理办法：把它**只当作 seed 的参考风格，不直接当训练目标**（训练目标由本周自己重新生成），或者做一次人工扫读并在 data card 里写明。这条判断本身值得写进 NOTES——真实产品的日志要变成训练数据，卡点从来不是技术，是"哪些字段绝对不能出去"。

任务 5：收工笔记（约 30 分钟）

NOTES 三行：今天最难决定的一处（多半是 T5 的标签来源）、明天第一条命令、明晚几点开始。

今日验收：gaps 三条有状态 + taxonomy 40 模板 + 每类金样例 + seeds.jsonl 500 行键集合断言通过 + PRIVACY.md。

完成判据：NOTES 三行齐全；`taxonomy.md` 与 `seeds.jsonl` 已 push（键集合断言通过的那段代码一并入库）。

Day 2 • 周二：先算账，再打通生成管线（200 条小样）

今日总览（6.5h）：今天要花的每一分钱都得先算过。**任务 1 是闸门：脚本里没有硬限额之前，一条 API 请求都不许发。** 之后打通生成管线（缓存、重试、seed 固定、提示版本化），产出 200 条小样并**人工看 30 条**——看不过就改提示重跑小样，别急着上量。今天 API 花费应控制在 ¥3 以内（200 条 + 演化 100 条）。

任务 1：API 算账与硬闸门（约 60 分钟）

1. 抄价：打开你今天核过的价格页，把**输入单价、输出单价（元/百万 token）**写进 `week07/cost.md`，标页面日期。**不确定就写区间，别猜。**
2. 实测用量：先发 20 条真实请求，统计每条的 `prompt_tokens` 与 `completion_tokens`（响应里的 usage 字段），取均值。
3. 外推并写死预算：

```
python - <<'PY'
IN_PRICE  = 0.0            # 元 / 百万输入 token，填你今天抄到的真实值
OUT_PRICE = 0.0            # 元 / 百万输出 token
IN_TOK, OUT_TOK = 0, 0     # 20 条实测的每请求平均 token，填实测值
N = 4000
cost = N * (IN_TOK * IN_PRICE + OUT_TOK * OUT_PRICE) / 1e6
print(f"单条 ≈ {(IN_TOK*IN_PRICE+OUT_TOK*OUT_PRICE)/1e6:.6f} 元 | 4000 条 ≈ ¥{cost:.1f}")
print(f"含 2 次重试的最坏情况 ≈ ¥{cost*2:.1f} | 本周上限 ¥30 -> {'通过' if cost*2 <= 30 else '必须缩小规模或换模型'}")
PY
```

4. 在生成脚本里加**硬闸门**：维护一个 `spend.json`（累计 `prompt_tokens/completion_tokens/估算金额`），每次请求前检查，超过 `MAX_SPEND_CNY` 就抛异常退出。**今天先测它**：把 `MAX_SPEND_CNY` 设成 0.5，跑 200 条，确认它会中途停下并打印停在哪一步。

完成判据：`cost.md` 有单价（带日期）、实测平均 token、外推金额、最坏情况四行；**闸门实测生效**（`MAX_SPEND_CNY=0.5` 那次运行确实报错退出，退出信息里含累计金额）；外推结论明确写出"4000 条在 ¥30 内可行 / 不可行，不可行则目标改为 N 条"。

> 注：三件事按顺序做，别颠倒：**先核价 → 再实测 20 条 → 再外推**。只看官方文档给的"平均一次几分钱"来预算是最常见的翻车方式，因为你的 prompt 里塞了金样例和 5 个 few-shot，实际输入 token 可能是文档例子的 5 倍。**降级路线**（按顺序试）：① 换更便宜的模型或更小的 max_tokens；② 把 4-5 条样本合并进一次请求（batch prompt，输入输出都省）；③ 用周 5/周 6 的租卡经验，在卡上跑 Qwen2.5-7B-Instruct 批量生成（¥8 卡时能出几千条，代价是质量下降、要多写推理代码）；④ 直接砍规模到 1500 条并同步改判据。

任务 2：生成脚本与缓存（约 150 分钟）

写 `scripts/generate.py`，五个硬性要求（缺一项判据不过）：

1. **提示词版本化**：`prompts/v1/T2.md` 这样的文件，脚本里 `PROMPT_VERSION="v1"`，输出文件名带版本（`gen_v1_001.jsonl`），**改一个字就升版本号**，别覆盖。

2. **随机性写死**：请求参数里 `temperature` 与 `seed` 固定（例如 `temperature=0.9, seed=42`），并在每条样本记录这两个值。**为什么**：可复现的语义是"同参数 + 同输入 → 可重跑"，温度不记就是不可复现，周 12 评测一致性抽查也依赖这条。
3. **按请求 hash 缓存**：`sha1(prompt_version + 渲染后的完整 prompt)` 作 key，已存在的 key 直接跳过 → **重跑不重复计费**。
4. 并发 ≤4，失败按 1s/3s/9s 指数退避重试 3 次，全失败落 `errors.jsonl`（含 HTTP 状态与响应体前 200 字）。
5. 每条样本记录：`task_type, seed_word, cefr, instruction, output, model, prompt_version, temperature, seed, usage`。

```
import hashlib, json, os, time
def cache_key(prompt_ver: str, rendered: str) -> str:
    return hashlib.sha1(f"{prompt_ver}|{rendered}".encode("utf-8")).hexdigest()

def already(cwd: str, key: str) -> bool:
    return key in _seen                    # _seen 从 cwd 下所有 gen_*.jsonl 的 key 字段加载
# 请求侧：requests.post(BASE_URL+"/chat/completions", headers=..., json={
#     "model": MODEL, "messages": [...], "temperature": 0.9, "seed": 42,
#     "max_tokens": 512}, timeout=60)
```

完成判据：跑 200 条（5 类各 40）**成功率 ≥90%**（失败数与原因写进 `errors.md`）；再跑一次同样 200 条，**新增 API 调用为 0**（打印 `cache hits: 200, requests sent: 0`）；`gen_v1_001.jsonl` 每行含上面 10 个字段（写断言）。

任务 3：人工检 30 条 + 格式校验（约 90 分钟）

1. 用 pydantic 定义 5 类输出的 schema，跑校验，打印合格率。
2. 随机抽 30 条（`random.seed(7)`）逐条看，给每条打 `ok / 需改 / 垃圾` 三档并写一句理由。
3. 定标准：**格式合格率 ≥80% 且"垃圾"档 ≤10%** 才允许明天上量；不达标今天就改提示重跑 200 条。

完成判据：两张数字（格式合格率、垃圾率）写进 `day2/review30.md`；30 条逐条理由齐全；若改过提示，写明改了哪一句、为什么。

> 注：这一步看起来"没产出"，实际是整周最省钱的一步。脏提示放大到 4000 条之后，你在 Day 4 的过滤器里删掉的会是一大半，等于白烧 ¥30。**看 30 条要 30 分钟，值得。**

任务 4：evol-instruct 两种演化实现（约 60 分钟）

对其中 100 条指令做演化，深度、广度各 100 条： - 深度：给口诀任务加约束（"只用 7 个字以内"、"必须含词根"）、要求对比两个词、要求先判难度再解释。 - 广度：同词换任务（释义 → 出题）、换角色（老师 → 学生来问）、换格式（JSON → 表格）、加多轮（追问一轮"再简单点"）。

完成判据：`evol_v1.jsonl` 200 行，每行有 `evo_op` 字段标操作类型；**人工看 10 对（演化前/后）且 ≥7 对认为"更难或更多样"**，10 对的判断写进 NOTES（不许只看一眼就打勾）。

任务 5：收工与花费回填（约 30 分钟）

`cost.md` 填今日实际（从 `spend.json` 抄）；`runs.csv` 加 3 行；把 `spend.json` 与 `gen_v1_*.jsonl` 的 key 列表 push（这是"重跑不重复计费"的证据）。

今日验收：闸门实测停过 + 200 条 ≥90% 成功 + 二次运行 0 请求 + 两张合格率数字 + 演化 200 条。

完成判据：`cost.md` 的实际金额与 API 账单页一致；`runs.csv` 三行（小样 / 缓存重跑 / 演化）；`spend.json` 与生成文件已 push。

Day 3 • 周三：全量生成 + 规则过滤 + PII 脱敏

今日总览（6.5h）：今天上量，也今天开始"删数据"。目标是收工前拿到一份**经过规则过滤与脱敏的中间集**，并且每条过滤规则都有明确命中数（**某条规则命中 0 条，多半是你写错了**）。API 花费今天预计 ¥20-25（含 Day 2 的 ¥3 左右，累计别超 ¥30）。

任务 1：全量生成 4000 条（约 150 分钟）

分批 4 × 1000，每批之间**停下来看账单和 20 条抽样**再继续（不要一次性挂完）。

```
python scripts/generate.py --target-seeds week07/day1/seeds.jsonl \
  --taxonomy dataset-english-tutor/card/taxonomy.md \
  --prompt-version v1 --n 1000 --batch-id 1 --temperature 0.9 --seed 42 \
  --out dataset-english-tutor/data/gen_v1_b1.jsonl
wc -l dataset-english-tutor/data/gen_v1_b*.jsonl
```

完成判据：`raw` 合并后行数 ≥3800（4000 目标 + 5% 容差）；`spend.json` 累计金额 ≤¥30 且与账单页一致（**不一致以账单为准，并在 cost.md 写一行原因**）；每批的 20 条抽检结论一句（有没有出现"整批同一种句式"这种问题，有就停下改提示）。

> 注：**别在晚上睡觉前挂全量**。API 超时、余额不足、格式漂移这类事都需要你当场判断要不要重跑；人不在的时候它会把 ¥20 花完并给你 4000 条垃圾。规则：任何一次 >¥5 的批量调用，必须人在屏幕前分批发。

任务 2：规则过滤器（约 120 分钟）

`scripts/rule_filter.py`，六条规则逐条实现并**各自统计命中数**：

规则	判据	为什么
长度	指令 <8 或 >600 字符；输出 <10 或 >1200 字符	太短没信息量，太长是模型跑偏
语言	中文字符占比与目标不符（T1/T2/T4 应含中文，T3 的 `en` 字段应基本纯英）	教学数据最容易语言配比失控

规则	判据	为什么
重复 n-gram	输出内 3-gram 重复率 >15%	兜住"复读机"式退化
截断	结尾无终止标点 / JSON 括号不闭合 / 缺必需字段	截断样本教模型输出残句
空与占位	值为 `""` / `null` / `"N/A"` /字段名自重复	模型偷懒的典型表现
敏感	命中 Day 3 任务 3 的 PII 正则	脱敏前置拦截

```
import json, re, collections
CJK = re.compile(r"[一-龠]")
def ngram_repeat_rate(s, n=3):
    g = [s[i:i+n] for i in range(len(s) - n + 1)]
    c = collections.Counter(g)
    return 1 - len(c) / max(1, len(g))
def zh_ratio(s):
    return len(CJK.findall(s)) / max(1, len(s))
```

完成判据：`--report` 打印每条规则的命中数与累计通过数，**六条规则每条命中 ≥1**（命中 0 的规则必须复查正则并写明结论）；**注入测试**：手工造 20 条坏样本（5 条复读、5 条截断 JSON、5 条空字段、5 条超长），过滤后**至少删掉 18 条**，两个数字都记进报告。

任务 3：PII 脱敏与人工抽检（约 60 分钟）

1. 正则脱敏（手机号、邮箱、身份证 15/18 位、银行卡 16-19 位、URL、QQ/微信号模式），替换成 `<PHONE>` / `<EMAIL>` / `<ID>` / `<BANKCARD>` / `<URL>` / `<ACCOUNT>`。
2. **中文姓名/地名不要用正则硬删**（误伤率太高）：改为白名单策略——种子池只有 word/cefr 两个业务字段，任何自由文本只允许来自"本次生成的输出"，而生成提示里明确不含用户信息。把这套逻辑写进 `PRIVACY.md` 的第 ② 版。
3. 人工抽检 100 条（`random.seed(11)`），逐条确认无个人信息残留。

```
import re
PII = [("<PHONE>", re.compile(r"1[3-9]\d{9}")),
       ("<EMAIL>", re.compile(r"[\w.+-]+@[\w-]+\.[\w.-]+")),
       ("<ID>",    re.compile(r"\d{17}[\dXx]|\d{15}")),
       ("<CARD>",  re.compile(r"\b\d{16,19}\b")),
       ("<URL>",   re.compile(r"https?://\S+"))]
```

完成判据：**注入测试**：20 条各含一类 PII 的样本，识别 **20/20** 全中（打印每类命中数）；100 条人工抽检记录（逐条 `OK/发现残留+原文片段`），**残留计数为 0**；`PRIVACY.md` 更新到 v2，写明"白名单 + 生成侧无用户信息 + 禁用表"三层。

> 注：真实数据脱敏的做法就是这三层：**能不带就不带（白名单字段）→ 必须带就规则化替换 → 替换完人工抽检**。别指望一个正则解决一切，也别指望"我们只用了词表所以没有隐私"——你要在 data card 里写清为什么这句话成立，它才对得起读者。

任务 4：中间集统计（约 30 分钟）

打印：原始行数、规则后行数、脱敏改写行数（不是删除而是替换，要单独计）、当前通过数、5 类任务占比、CEFR 分布。写进 `day3/mid-stats.md`，这是 data card 里"清洗规则命中数"那张表的原材料。

完成判据：五个数字齐全且与脚本输出一致（**不许手填**，把打印重定向到文件）。

任务 5：收工笔记（约 30 分钟）

回填 cost；runs.csv 加行；把 `raw_4k.jsonl` 备份一份（网盘或卡上持久盘）——**过滤链可以重跑，原始数据丢了就再也花不回那 ¥30**。

今日验收：raw ≥3800 + 六条规则命中表 + 注入 18/20 + PII 20/20 + 人工抽检 100 条零残留。

完成判据：`raw_4k.jsonl` 有第二份备份（网盘或卡上持久盘），备份路径写进 `COSTS.md`；今日与累计花费回填。

Day 4 • 周四：四板斧后两板斧——MinHash 去重、困惑度过滤、RM 打分

今日总览（6.5h）：今天是本周技术密度最高的一天，也是你以后每次做数据都要抄的一天。**关键交付物不是分数，是 `pipeline.sh`：给定 `raw_4k.jsonl` 能重跑出完全相同的最终集（md5 一致）。** 卡时今天 2.5h（PPL + RM），留 1.5h 机动。

任务 1：MinHash-LSH 近重复去重（约 100 分钟）

```
from datasketch import MinHash, MinHashLSH
def shingles(s, k=5):
    s = s.strip()
    return {s[i:i+k] for i in range(max(1, len(s) - k + 1))}
def mh(s, num_perm=128):
    m = MinHash(num_perm=num_perm)
    for sh in shingles(s):
        m.update(sh.encode("utf-8"))
    return m
lsh = MinHashLSH(threshold=0.85, num_perm=128)
kept, dropped = [], 0
for i, r in enumerate(rows):
    text = r["instruction"] + "\n" + json.dumps(r["output"], ensure_ascii=False)
    key = f"d{i}"
    if lsh.query(mh(text)):
        dropped += 1
        continue
    lsh.insert(key, mh(text)); kept.append(r)
print(f"kept={len(kept)} dropped={dropped} rate={dropped/(len(kept)+dropped):.2%}")
```

完成判据：打印 kept/dropped/去重率（**预期量级 5%-20%**，超 30% 说明阈值太松或 Day 2 的演化没做够，回去看句式分布而不是直接改阈值）；**注入测试**：造 20 条"只改 3-5 个字"的近重复，脚本必须删掉 **≥16** 条；阈值从 0.85 调到 0.95 时去重率的变化写一句话（这题周 14 八股会问）。

任务 2：困惑度过滤（约 120 分钟，卡时 1.5h）

用 **Qwen2.5-0.5B（base，非 instruct）** 给每条样本的输出段算 PPL。为什么要上卡：4000 条 × ~200 token 的前向在 CPU 上要以小时计。**双向截断**：过高是噪声/乱码，过低是模板复读（模型觉得"显然会说这句"，通常意味着它没提供信息）。

```
import torch, torch.nn.functional as F
from transformers import AutoModelForCausalLM, AutoTokenizer
M = "/root/autodl-tmp/models/qwen05b"
tok = AutoTokenizer.from_pretrained(M)
model = AutoModelForCausalLM.from_pretrained(M, torch_dtype=torch.float32,
                                             device_map="cuda").eval()   # 统一 fp32：bf16 下 CE 会漂
@torch.no_grad()
def ppl_of(text):
    ids = torch.tensor([tok(text, add_special_tokens=False, truncation=True,
                            max_length=512)["input_ids"]], device=model.device)
    if ids.shape[1] < 2:
        return float("inf")
    logits = model(ids).logits[:, :-1]
    loss = F.cross_entropy(logits.reshape(-1, logits.shape[-1]).float(),
                           ids[:, 1:].reshape(-1), reduction="mean")
    return float(torch.exp(loss))
```

批量跑法：`batch=16`、按长度排序减少 padding、进度落 `ppl.jsonl`（`word, ppl`）可断点续跑。**先只看 500 条决定区间**，画直方图，再定 `[P10, P90]` 与绝对上限。

完成判据：`ppl_hist.png` 入库（横轴用对数刻度，否则全挤在左边）；**双向各删掉的条数打印出来**；**注入测试**：20 条"随机 token 拼出的噪声文本"，其中 **≥19 条**落在高 PPL 侧被删；写一句你的区间选择的理由（不许写"经验值"）。

> 注：PPL 过滤有个天生的偏差：**基座模型偏爱的文体 PPL 低**，于是你会悄悄把"最像 Qwen 说法"的样本留下、把多样表达删掉。所以今天必须双向截断，并在 data card 的"已知偏差"里老实写这一条。面试官问"困惑度过滤有什么问题"，这就是答案，也是你亲手做过才有的答案。另外：**别用 instruct 模型算 PPL 来筛数据**——它的输出分布已经被 SFT 塑过，会系统性偏袒 SFT 腔。

任务 3：奖励模型打分筛选（约 100 分钟，卡时 1h）

用一个公开 RM 对每条 `(instruction, output)` 打分，删最低的一档（先删 15%）。

```
from transformers import AutoModelForSequenceClassification, AutoTokenizer
RM = "OpenAssistant/reward-model-deberta-v3-large-v2"     # 下载走 HF_ENDPOINT 镜像
tok = AutoTokenizer.from_pretrained(RM)
rm = AutoModelForSequenceClassification.from_pretrained(
        RM, torch_dtype=torch.bfloat16).to("cuda").eval()
@torch.no_grad()
def score(instr, resp):
    x = tok(instr, resp, truncation=True, max_length=1024, return_tensors="pt").to("cuda")
    return float(rm(**x).logits[0, 0])
```

降级路线（按顺序）：① 下载失败/显存不够 → 换更小的 RM；② 都不行 → **LLM-as-judge 打分**：同一段评审提示打 3 次（temperature=0），取中位数，**极差 ≥2 分的样本进人工队列**；③ 本周只跑 1000 条子集的 RM 分数并说明覆盖比例。

完成判据：分数直方图 + 阈值（含分位数）+ 删除条数入库；**人工抽 20 条低分样本，判断"确实差"的比例 ≥70%**（写成一张 20 行表：样本摘要 / RM 分 / 你的判断 / 理由）；一致性抽查：任取 10 条**重跑 3 次打分，极差为 0**（确定性推理；非 0 就要写清原因）。

> 注：RM 打分当过滤器用要小心它**偏爱长回答和客套话**——这正是 reward hacking 的前置知识（周 8/周 9 会正式撞见）。所以你今天的正确姿势是：**RM 分数只做下限剔除（删最差的 15%），不做排序优化**，并在 NOTES 记一句"为什么不用它排 top-k"。这一句在面试里的分量比过滤本身重。

任务 4：一键复现：`pipeline.sh` 与两次同 md5（约 50 分钟）

```
set -euo pipefail
cd "$(dirname "$0")/.."
python scripts/rule_filter.py   --in data/raw_4k.jsonl   --out data/f1_rule.jsonl
python scripts/pii_scrub.py     --in data/f1_rule.jsonl  --out data/f2_scrub.jsonl
python scripts/dedupe.py        --in data/f2_scrub.jsonl --out data/f3_dedup.jsonl   --threshold 0.85
python scripts/ppl_filter.py    --in data/f3_dedup.jsonl --out data/f4_ppl.jsonl     --lo-pct 10 --hi-pct
90
python scripts/rm_filter.py     --in data/f4_ppl.jsonl   --out data/f5_rm.jsonl      --drop-pct 15
python scripts/decontaminate.py --in data/f5_rm.jsonl    --out data/final.jsonl      --against
eval/gsm8k_test.txt eval/eval20.jsonl --ngram 13
sha256sum data/final.jsonl | tee data/final.sha256
```

完成判据：`bash scripts/pipeline.sh` 跑通，**连续两次运行的 `final.jsonl` sha256 完全一致**（不一致就逐个中间文件比对定位是哪一步不确定——多半是 `set` 遍历顺序或时间戳字段没固定，把定位过程写进 NOTES）。

任务 5：收工（约 20 分钟）

回填 COSTS（今天计划 2.5h，累计 ≤4h）；关机确认；四份过滤报告存 `dataset-english-tutor/card/reports/`。

今日验收：去重注入 ≥16/20 + PPL 双向数字与注入 ≥19/20 + RM 抽检 ≥70% 且一致性 10/10 + pipeline 两次 md5 相同。

完成判据：今天 ≤2.5h、本周累计 ≤4h 已填 `COSTS.md`；控制台显示「已关机」；`card/reports/` 下四份报告存在且非空。

Day 5 • 周五：去污染、分布图与 data card

今日总览（6h）：今天把数据集从"一堆文件"变成"一份能被别人信任的公开数据"。三件事：**去污染（不许有评测集原文）**、**统计与分布图**、**data card**。data card 不是格式作业——周 12 你要用它证明"我知道我的数据有什么问题"，而面试里"你的数据是怎么来的"这题，答不出偏差的人一追问就空。

任务 1：去污染（约 120 分钟）

对照集两份：**周 6 冻结的** `eval20.jsonl` **（含 forget10）**、以及 **GSM8K 的 test 划分文本**（周 9 要在它上面做 GRPO，训练数据绝不能含它；下载走 `export HF_ENDPOINT=https://hf-mirror.com` + `load_dataset("openai/gsm8k","main")` 的 test split）。**任何评测集以任何形式（原文、改写、翻译）进训练集都不许。**

```
def norm(s):
    import re
    return re.sub(r"\s+", " ", s.lower().strip())
def ngrams(s, n=13):
    w = norm(s).split()
    return {tuple(w[i:i+n]) for i in range(max(1, len(w) - n + 1))} if len(w) >= n else set()
def contam_hit(cand_words_ngrams, eval_sets):
    for tag, grams in eval_sets.items():
        inter = cand_words_ngrams & grams
        if inter:
            return tag, next(iter(inter))
    return None, None
```

短样本（不足 13 词，例如单词分级题）**降级用字符级 8-gram 交集**，并单独统计这一档命中数——别用 13 词去卡一个单词，那等于没做去污染。

完成判据：**注入测试**：把 10 条 GSM8K test 原文塞进候选集，脚本必须 **10/10 全部剔除**；正式剔除数（预期量级 0-5 条）与对照集规模写进报告；`final.jsonl` 重跑 pipeline 后与 `eval20` 的 13-gram 交集为 **空集**（打印 `DECONTAM OK`）。

> 注：13-gram 是社区常用的粗筛（不同团队用 8/10/13 都有），它挡不住改写和翻译，所以你要在 data card 里如实写"去污染手段 = 词级 13-gram + 短样本字符 8-gram，**未做语义去污染**"。面试官问"你怎么防评测污染"，"n-gram 交集 + 明确写出它挡不住什么"比"我做了去污染"强得多。

任务 2：分布统计与图（约 90 分钟）

```
python - <<'PY'
import json, collections
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
rows = [json.loads(l) for l in open("dataset-english-tutor/data/final.jsonl", encoding="utf-8")]
il = [len(r["instruction"]) for r in rows]; ol = [len(json.dumps(r["output"], ensure_ascii=False)) for r
in rows]
tt = collections.Counter(r["task_type"] for r in rows)
cf = collections.Counter(r.get("cefr", "?") for r in rows)
src = collections.Counter(r.get("source", "gen") for r in rows)      # gen / evol-deep / evol-broad
fig, ax = plt.subplots(2, 2, figsize=(10, 7))
ax[0,0].hist(il, bins=40); ax[0,0].set_title("instruction length (chars)")
ax[0,1].hist(ol, bins=40); ax[0,1].set_title("output length (chars)")
ax[1,0].bar(list(tt), [tt[k] for k in tt]); ax[1,0].set_title("task type")
ax[1,1].bar(list(cf), [cf[k] for k in cf]); ax[1,1].set_title("CEFR level")
fig.tight_layout(); fig.savefig("dataset-english-tutor/figures/dist.png", dpi=120)
stats = {"n": len(rows), "instr_len_p50_p90_p99": [int(x) for x in
         [sorted(il)[len(il)//2], sorted(il)[int(len(il)*.9)], sorted(il)[int(len(il)*.99)]]],
         "output_len_mean": round(sum(ol)/len(ol), 1),
         "task_dist": dict(tt), "cefr_dist": dict(cf), "source_dist": dict(src)}
json.dump(stats, open("dataset-english-tutor/card/stats.json", "w", encoding="utf-8"),
          ensure_ascii=False, indent=2)
print(json.dumps(stats, ensure_ascii=False))
PY
```

完成判据：`figures/dist.png` 四宫格齐；`stats.json` ≥8 个字段且**图上的数字与 json 里的一致**（自己核对一遍，不一致就是脚本有两条路径，修掉它）。

任务 3：data card（约 120 分钟）

`dataset-english-tutor/card/README.md`，十一个小节（**这就是发布时的 README**，一项不缺）：

1. 一句话用途与许可（**数据许可与代码许可分开写**；本数据集由 LLM 生成，许可建议 CC BY-NC 4.0 并注明用途限制）
2. 规模与构成（引用 `stats.json` 的具体数字，不许写"数千条"）
3. **生成模型与提示**：模型名、`prompt_version` 对应文件、temperature/seed、API 单价与**真实总花费**
4. 种子来源：词库（许可）+ 明日记忆历史查询（**只取 word/level 两列，已去标识化**）
5. 清洗规则与每步命中数（六条规则 + 去重 + PPL + RM + 去污染，一张表）
6. 长度/主题/难度分布（贴 `dist.png`）
7. **已知偏差（≥3 条，不许写"无"）**：候选写法：Qwen 系基座的 PPL 偏置；RM 偏爱长回答；T5 的 CEFR 标签来自单一词典（同一词在不同词典可能不同级）；题材集中在中国学生学英语的场景
8. 使用限制：不用于评测之外的用户画像；不含真实用户自由文本
9. 复现：`bash scripts/pipeline.sh` + `sha256` + `requirements.txt`
10. 维护：版本号 `v0.1`、如何提 issue、后续会把周 8 的偏好对另发一版

11. 引用与联系方式

完成判据：11 小节齐全；第 5 项的表格数字与 Day 3/Day 4 的报告一致（**逐格核对，对不上就回去改报告而不是改这张表**）；第 7 项 ≥3 条且每条都说清"什么机制导致它"。

任务 4：收工（约 30 分钟）

NOTES ≥5 条（今天多半是"发现两处数字对不上"的记录）。

今日验收：注入 10/10 剔除 + `DECONTAM OK` + dist.png + stats.json ≥8 字段 + data card 11 节齐全。

完成判据：NOTES ≥5 条，其中至少 1 条是「两处数字对不上」的定位过程；`card/README.md` 已 commit。

Day 6 • 周六：发布 + 复现包 + 第 2 篇博客

今日总览（6h）：今天把东西交出去。三件：数据集在 ModelScope（有余力再补 HF）、一个别人能跑通的复现包、专栏《从全栈工程师到后训练》的**第 2 篇博客**（本周配额，以 data card 为主体写成实操帖）。发布前**再读一遍 PRIVACY.md 与许可小节**——公开之后删不干净，这是本周真正的底线。

任务 1：ModelScope 发布（约 90 分钟）

1. 魔搭控制台 → 创建数据集，命名含 `english-tutor-instruct`（例如 `<你的用户名>/english-tutor-instruct`），填许可与简介。
2. 上传（命令以 `modelscope upload --help` 为准，参数顺序/可选值以实测为准）：

```
pip install -U modelscope
modelscope login --token <你的SDK令牌>
modelscope upload <你的用户名>/english-tutor-instruct ./dataset-english-tutor/upload \
  --repo-type dataset
```

3. 上传内容：`data/final.jsonl`、`card/README.md`（作为数据集主页）、`card/stats.json`、`card/PRIVACY.md`、`scripts/`、`requirements.txt`、`figures/dist.png`。**不要上传** **`raw_4k.jsonl`**（含未经完整复核的原始生成内容，且体积大）；data card 里写明"原始样本可按需提供"。

完成判据：链接**用未登录的浏览器（或无痕窗口）能打开并下载到 jsonl**；数据集主页 README 渲染正常（表格与图片不破）；文件列表含 `final.jsonl` 且行数与本地一致（页面能看文件大小，本地 `wc -l` 对得上）。

任务 2：Hugging Face 补传（约 30 分钟，可砍项）

```
export HF_TOKEN=<你的读写作 token>
pip install -U "huggingface_hub[cli]"
huggingface-cli upload <你的用户名>/english-tutor-instruct ./dataset-english-tutor/upload \
  --repo-type dataset
```

完成判据：HF 链接可公开访问；**若失败**（网络、token 权限、需要邮箱验证）：把失败的具体报错原文写进 `gaps.md`，标注"推到周 9 补，届时仍失败就只保留 ModelScope 并在博客里写明"，**不许在链接不存在时把 HF 链接写进任何文档**（面试诚信）。

> 注：`hf-mirror.com` **是只读镜像，不能上传**，这是最容易被想当然的地方。上传要么直连 huggingface.co，要么走你自己配置的科学网络；卡上试比本地试更容易成功（实例的网络出口不同）。试 30 分钟不成就是可砍项——ModelScope 那份已经满足"公开数据集"这个交付定义。

任务 3：复现包与干净环境冒烟（约 90 分钟）

1. `dataset-english-tutor/upload/` 里保证：`requirements.txt`（钉版本）、`make_dataset.sh`（从 `raw` 到 `final` 的一键链）、`README` 三条命令（装依赖 → 跑 pipeline → 校验 sha256）。
2. **冒烟测试**：新建一个干净 venv，只按 README 的步骤跑 **100 条子样**（`data/sub100.jsonl`），确认结果与仓库里那份 100 条子集的 sha256 一致。
3. 生成路径里的绝对路径全部改成相对/参数（`grep -rn "E:/" scripts/` 清零才算过）。

完成判据：干净 venv 里三条命令跑通、100 条子集 sha256 一致；`grep -rn "E:/study" scripts/` **无输出**；`requirements.txt` 里每个包都有版本号。

任务 4：博客第 2 篇（约 120 分钟）

标题方向：《我造了 4000 条教学对话数据，然后删掉了 1200 条》（**数字用你的真实数字**）。2000-2800 字，结构：

1. 为什么后训练岗一半时间在干数据（不是练模型，是"数据决定上限"这句话的证据）
2. 场景与 taxonomy：明日记忆的五类教学任务，T5 为什么可以自动验证（点一句"这就是周 9 规则奖励的种子"）
3. 合成：self-instruct 起步 + evol 深/广度，附演化前/后 2 个对比例子（真实）
4. **四板斧**：去重 / 困惑度 / RM / 规则，每一步的**真实命中数表**，加一句"为什么 RM 只做下限剔除"
5. 脱敏与合规：三层做法、禁用 `chat_messages` 的判断过程、许可怎么定
6. 去污染：为什么现在就要把 GSM8K test 挡在外面（三周后才用得上，这就是提前量的意义）
7. 已知偏差三条（照 data card，别藏）
8. 数据集链接 + 复现命令 + 下一篇预告

完成判据：正文 ≥2000 字、图 ≥2 张（`dist.png` + 任一过滤前后对比例子）、**表格里的每个数字都能在报告里找到出处**、博客里明写"数据由 LLM 合成，含偏差，见 data card 第 7 节"（不许把合成数据写成"真实数据"）。

任务 5：收工（约 30 分钟）

两个链接（数据集、博客）贴 `NOTES.md` 顶部；`runs.csv` 收口；COSTS 汇总（API 实际 / 卡时实际）。

今日验收：ModelScope 链接无痕窗口可访问 + 复现包冒烟 sha256 一致 + 博客已发布（有链接）+ HF 已传或有失败原因记录。

完成判据：`NOTES.md` 顶部两个链接（数据集、博客）能点开；`COSTS.md` 本周三项（API 花费 / 卡时 / 真实时间）全部填完。

Day 7 • 周日：补漏 + 闭卷复测 + 本周总验收 + 周 8 预习

今日总览（4.5h）：不开卡、不打付费 API。今天收口：两个洞、一次闭卷、一张验收表、40 分钟预习周 8（**周 8 的偏好对就从现在开始挂钩**）。

任务 1：补漏（约 90 分钟）

打开 `gaps.md` 与四份过滤报告，**最多挑 2 个洞**补。优先级：闭卷要考的四板斧原理 > pipeline 不确定步骤 > HF 上传。补完在 gaps 里写"怎么补的 + 一句证据"。

完成判据：最多 2 个洞，每个留一条证据（重跑命令 + 输出数字）；对应报告文件已更新，而不只是在 gaps 上打勾。

任务 2：闭卷复测（约 60 分钟）

新文件 `week08/test-closedbook.md`，限时 25 分钟写完，再对答案：

1. 四板斧各自**抓的是哪一类脏数据**？各写一句原理 + 一条它的失效场景（去重抓不到同义改写；PPL 有基座文体偏置；RM 偏爱长回答；规则抓不住"格式对但内容错"）。
2. 手写 13-gram 去污染的判定逻辑（伪代码 ≤12 行，含归一化与短样本降级）。
3. 深度演化与广度演化差别一句话，并各给一个你实际用过的操作。
4. 合成数据要变成可发布数据集，除了过滤还需要哪三样东西？（data card / 许可 / 隐私与用途限制）
5. 你的数据集的三个已知偏差（不许翻卡片）。

完成判据：5/5 写满，第 1 题的"四板斧各一句原理 + 失效场景"**全部写对**；错点当场写进 gaps 并排到周 8 Day 7 复测。

任务 3：本周总验收（约 40 分钟）

#	验收项	判据
1	选题与 schema	5 类 ≥40 指令模板；每类 1 条手写金样例；T5 标签来自词库而非模型
2	生成管线	硬闸门实测生效（设 ¥0.5 会中断）；200 条重跑 0 请求（hash 缓存）；seed/temperature 入库
3	规模与花费	`raw` ≥3800 条（或降级后 ≥1400 且判据同步改过）；API 实际 ≤¥30 且与账单一致
4	四板斧	规则 6 条命中表 + 注入 18/20；去重注入 ≥16/20；PPL 双向 + 注入 ≥19/20；RM 人工抽检 ≥70% + 重跑极差 0
5	合规	20/20 PII 注入识别 + 100 条人工抽检零残留 + PRIVACY.md v2 + `DATA-SOURCES.md` 许可无空

#	验收项	判据
6	去污染与可复现	GSM8K 注入 10/10 剔除 + `DECONTAM OK` + `pipeline.sh` 两次 sha256 一致 + 干净 venv 冒烟通过
7	公开输出	ModelScope 链接无痕可访问、行数与本地一致；data card 11 节齐全含 ≥3 条已知偏差；博客第 2 篇已发布

≥6 项通过 = 本周合格（数据集这一件 M2/M3/M4 的原料到位）；不足 6 项，缺的写进周 8 Day 1 前 90 分钟，并标注"是否阻塞周 10 的 M4 数据准备"。

完成判据：表格 7 行都有勾选或写明缺项与补齐日期；`final.jsonl` 的 sha256 与发布页上的文件行数/大小一致。

任务 4：新暴露的洞 → 后续周映射（约 40 分钟）

`week08/gaps-from-week7.md`，≥4 条，**必须落到周 8/周 9/周 10-12**：

洞	归到	动作
只做过"一条指令一条回答"，没做过偏好对	周 8 Day 1-2	用本数据集的 prompt 对周 6 模型采 3-4 个输出 → 打分 → max-min 配对 1000-2000 对，并做 100 条人工复核与一致率
RM 分数只会用来删下限，不会当奖励	周 9	GRPO 的规则奖励（T5 的 CEFR 分类对不对 + 输出 JSON 合法性）+ RM 分数作对照
合成数据没有 held-out 评测子集	周 8 切 held-out → 周 12 扩到 200 题	从 `final.jsonl` 按 task_type 分层抽 200 条，**打 tag 冻结并从训练集剔除**
许可与可再分发只做了粗判	周 12 模型上传前	复核词库许可（`ec-cedict` / `stardict` 的许可条款）并在模型卡里一并声明
分布偏斜（某类任务/某等级样本少）	周 10 M4 数据配比	按 T1-T5 与 CEFR 做重采样或补生成，写清配比依据

完成判据：≥5 行且每行有动作与日期；"held-out 切分"这条必须在周 8 Day 1 之前完成（否则周 8 的评测没有干净数据）。

任务 5：周 8 预习 + 本周对账（约 40 分钟）

1. 读 DPO 论文的 loss 那一节（不看推导细节），把 Bradley-Terry 的损失抄一行进 NOTES，并写一句"它和 RM 有什么关系"。
2. 想清楚一件事并写 100 字：**周 7 的 RM 打分脚本，怎么改造成周 8 的偏好标注器？**（提示：现在是"给单条打分"，偏好对要的是"同一 prompt 下多个输出比高低"，那就要把同 prompt 的多个输出放进同一批打分、取 max-min 配对。）
3. 对账：`COSTS.md` 汇总本周（API 计划 ¥30 / 实际 ?；卡时计划 4h / 实际 ?；时间真实花销按天记，对照 42h）。

完成判据：BT 损失一行 + 100 字答案 + 对账三项填完。

今日验收：验收表 ≥6 项打勾 + 闭卷 5/5 + gaps 表 ≥5 行 + 对账完成。

三、常见问题与纪律

- **API 花超了怎么办？** 立刻停（改 `MAX_SPEND_CNY=已花金额`，让脚本自己拒跑），然后按总览降级：**规模从 4000 降到 1500，判据同步改写并在 data card 里写明**。永远不许为了"凑够 4000"去放宽过滤阈值——那等于把 ¥30 换成一份更差的公开数据，还带偏差。
- **过滤器太狠，剩不下几条？** 先看命中分布：某条规则删掉一半以上通常是阈值写错（尤其中文场景按英文假设写的长度、重复率）。修阈值的依据必须是**你人工看了被删的 20 条**，不是"感觉太狠"。每一次阈值调整都在 data card 的第 5 节留一行，别偷偷改。
- **本地跑不动 / 卡抢不到？** 本周三件事里只有 PPL 与 RM 需要卡。PPL 可降级为"只算 1000 条子集并说明覆盖比例"；RM 可降级为 LLM-as-judge（走 API，但把评审请求算进 ¥30 预算）。**去重、规则过滤、去污染、发布全部不需要卡**——不许用"没卡"当这些环节的借口。
- **合规复查（发布前后各一次）**：发布前逐条确认——① 每条种子字段都在 `PRIVACY.md` 白名单里；② 词库的许可条款抄进 data card（含是否允许再分发）；③ 数据集里没有从周 6 那份 `CC-BY-NC` 指令集抄来的任何文本；④ data card 第 3 节写的是真实模型名和真实提示版本，不是"某大模型"。发布后发现自己漏了字段，**立刻删库重传并在博客补一条更正说明**，比被人发现体面得多。
- **别把这份数据集当成"够了"。** 它是 M2/M3/M4 的原料：周 8 的偏好对、周 9 的可验证任务、周 10 的 SFT 数据都从这里出。所以它的 `task_type` / `cefr` 字段设计要能一路支撑到周 12——今天多写一行字段说明，周 11 少写一个转换脚本。
- **AI 可以做什么**：帮你想演化操作、写正则、审 data card 的措辞。**不能做**：替你判断 30 条小样好不好、替你写金样例、替你填过滤报告的命中数。周 15 项目深挖会问"你那批数据最难删的是哪一类"，只有你真看过才答得出。